

When Coding Agents Say "Done" but Ship Broken Code — and What Measurably Fixes It

Executable acceptance feedback in frontier coding agents at brownfield scale: a two-lineage controlled replication (bounded)

*Working draft v2 — 2026-06-23. Preliminary, controlled two-arm evidence now **replicated across two independent model lineages** (DeepSeek V4 Pro and Alibaba Qwen 3.7 Max), plus single-condition cross-lineage corroboration from production tools; bounded to this task family / substrate / budget — not a general law (see §7). Written for both research and engineering-leadership readers — §"The short version", §5 (corroboration), and §6 (what it means for leaders) speak to the latter; §2–4 and the appendix carry the method and statistics.*

What this evidences (and what it doesn't). The manipulated variable is **executable acceptance feedback**: whether the agent can run the (pre-existing) hidden acceptance oracle on each loop as its stopping condition. That is the *operational core* of **Harnessed Iterative Spec-Driven Development (HIT-SDD)** — but **not** the full practice: these runs use *existing* acceptance tests, not an *authored* spec (OpenSpec/Gherkin). So HIT-SDD here is the **narrative frame / future direction**, never an experimental condition; an authored-spec study is pre-registered (§8).

The short version (for the busy reader)

- **The problem.** AI coding agents routinely declare a task *done* while the change still fails its acceptance tests — they ship broken work *confidently*. On real codebases that means confidently-wrong patches reaching review and merge.
- **The experiment.** We changed exactly one thing about an otherwise-identical frontier coding agent: whether it could **run the acceptance tests** (without seeing the expected answers) before declaring done.
- **The result, now on two independent models.** Letting the agent run the tests **near-eliminates confidently-wrong shipping** on real, post-cutoff brownfield tasks:
 - **DeepSeek V4 Pro:** "declares done while actually broken" fell **79% → 13%**, and the fix rate **roughly doubled (19% → 38%)** (n=9; 8/9 tasks significant).
 - **Qwen 3.7 Max (independent replication):** the same gap fell **50% → 0%** (n=9; 5/9 tasks significant; treatment shipped *zero* confidently-wrong patches on every task).

- **The takeaway for builders.** The cheapest reliability upgrade for an agentic coding tool may not be a bigger model. It is **giving the agent an executable definition of "done" and making it run that before it claims success** — Harnessed Iterative SDD.
- **It's not one model's quirk.** Beyond the two controlled models, the production tools developers use today — **OpenAI Codex (GPT-5.5)** and **Anthropic Claude Code (Opus 4.8)** — show the *same* confident-but-broken shipping on these tasks, **even though they can already run tests themselves**. Running your own tests is not the same as running the acceptance contract.
- **The honest caveat.** Two independent lineages now agree — this is past a single-model fluke — but the effect is **bounded** to one task family, one substrate (SWE-bench Live, Python brownfield), one scaffold, and one budget. It is **not yet a general law**; the magnitude is model-dependent (smaller on the stronger self-verifier), and patch-level replay is pending a harness improvement (§7).

Abstract

Agentic coding tools frequently declare a task complete while the change does not satisfy its acceptance criteria — they ship broken work with confidence. We ask whether giving a frontier coding agent the ability to **execute** a hidden acceptance test suite (without revealing expected values) reduces this failure at realistic brownfield scale — the operational core of *Harnessed Iterative Spec-Driven Development* (HIT-SDD). We run a controlled, two-arm ablation on real post-training-cutoff GitHub issues (SWE-bench Live), holding task, repository, agent scaffold, and compute budget fixed and manipulating one variable: whether the agent can run the acceptance oracle. Our primary outcome is the **self-verification gap** — the agent declares done while the hidden oracle would fail. On $n = 9$ contamination-screened, flake-certified tasks (10 runs/arm/task), executable feedback reduces the gap from **79% to 13%** with **DeepSeek V4 Pro** (8/9 tasks significant; family-wise null probability $\approx 3.4 \times 10^{-10}$; resolve rate $\approx$ doubled, 19%→38%) and, in a pre-registered **independent-lineage replication** with **Qwen 3.7 Max**, from **50% to 0%** (5/9 tasks significant; family-wise null probability $\approx 3.3 \times 10^{-6}$; treatment self-verification gap of **zero on every task**). The benefit follows a difficulty gradient — redundant on easy tasks, generative on solvable ones, diagnostic (preventing false confidence) on tasks beyond the model's reach — and that gradient is **model-dependent**: the stronger self-verifier (Qwen) has lower control gaps and shows a **diagnostic-dominant** effect (resolve approximately flat, 37%→36%), while the weaker self-verifier (DeepSeek) shows both diagnostic and generative gains. As cross-lineage corroboration (single-condition calibration, not a controlled arm), the production CLI agents **Codex (GPT-5.5)** and **Claude Code (Opus 4.8)** exhibit the same gap on these tasks **despite executing their own tests** — landing in the no-execution regime, not the acceptance-oracle regime. The contrast is

acceptance-execution vs. no-execution (not vs. agent-written tests). Two independent lineages now agree; the result is **bounded** to this task family, substrate, scaffold, and budget — not a general claim.

1. Why this matters

The expensive failure of today's coding agents is not that they fail — it is that they fail *confidently*. An agent edits some source, announces the bug is fixed, and stops, while the acceptance tests are still red. A loud failure gets caught; a confident one gets reviewed, trusted, and merged. As these agents move from toy snippets to large existing ("brownfield") codebases, that false confidence is the reliability problem teams actually hit.

There is an obvious-sounding remedy: wire an **executable definition of "done"** into the agent's loop — let it run the acceptance criteria and keep going until they pass. This is, in plain terms, **Harnessed Iterative Spec-Driven Development (HIT-SDD)**: the behaviour-driven / specification-driven idea, but *operational* — acceptance criteria the agent *executes* on each iteration, not prose it reads once. The open question is whether this genuinely helps a *frontier* model, or whether such models already check themselves well enough that running tests adds nothing.

This paper answers that question with a controlled experiment that changes exactly one variable, and replicates it on a second, independent model lineage.

2. How we tested it

(In one line: same agent, same task, same repo — the only difference is whether it can run the acceptance tests. Everything below is to make that comparison trustworthy.)

Real, recent tasks. We use actual GitHub issues from **SWE-bench Live**, restricted to those created *after the model's training cutoff* (so the model can't have memorized the fix), that touch **multiple source files** (real regression risk), on repositories whose test suites are large enough to be meaningful but small enough to run reliably.

We checked the model hadn't memorized them — for each model separately. Popular repositories are heavily represented in training data, and "post-cutoff" is model-relative. So for each model we ran a **memorization probe**: give the model the exact start of a changed code region and ask it to continue — high verbatim overlap means recall, not reasoning. A *Zen of Python* positive control scored a perfect match; the actual tasks scored ≈ 0 for **both** DeepSeek and Qwen. No memorization detected for either lineage.

We proved the test oracle is trustworthy. A flaky test would invalidate everything. So each task's suite was run **60 times** on the reference fix; we admitted only tasks whose results were stable (flaky tests quarantined). This certifies the "did it pass?" judgment is deterministic, and it is model-independent (it carries across both models).

The one manipulated variable. Both arms use the *same* agent (OpenHands) on a sanitized copy of the repo (future git history removed; no network at run time), same prompt, same effort budget. The difference:

- **Control** — the agent can read and edit files, but **cannot run anything**.
- **Treatment** — the same, **plus one tool: it can run the hidden acceptance tests and see pass/fail per check** (it never sees the expected values).

What we measured. For each run we record whether the agent **declared "done,"** then score its final patch against the hidden tests ourselves (storing the per-test pass/fail outcomes and the patch's hash — the patch *text* was not retained; see §7). The headline metric, the **self-verification gap**, is the rate of runs where the agent *declared done but the tests would actually fail* — i.e., confidently-wrong shipping. Our secondary metric is the **fix rate** (all the bug's target tests pass). We ran **10 attempts per arm per task** on **DeepSeek V4 Pro** and, as a pre-registered replication, on **Qwen 3.7 Max** (independent lineage).

How we judged significance. For each task we used a **permutation test** (distribution-free — it asks how often random label-shuffles would produce an effect this large). We combined across the 9 tasks with a **family-wise** correction and required each "win" to clear a minimum meaningful effect of 0.20. By a rule fixed in advance, a single model is a *candidate*; a positive on a **second, independent lineage** is the pre-registered bar for a **replicated** (still bounded) finding.

3. What we found

3a. DeepSeek V4 Pro (original)

8 of the 9 tasks showed a statistically significant benefit; the combined probability under "no real effect" is $\approx 3.4 \times 10^{-10}$.

Task	"Done but broken" — control	— treatment	Reduction	Significance (p)
twine-1249	100%	0%	−100 pts	<0.0001
django-guardian-899	100%	10%	−90	<0.0001
datamodel-code-generator-2408	90%	0%	−90	0.0001
datamodel-code-generator-2461	100%	10%	−90	0.0002
drf-json-api-1283	70%	0%	−70	0.0021
codecarbon-831	100%	50%	−50	0.014
kombu-2300	100%	50%	−50	0.018
pycasbin-392	50%	0%	−50	0.017
freezegun-582	0%	0%	0	(no effect — easiest task)

Across all runs: ships "done" but broken **79% → 13%**; actually fixes the issue **19% → 38%**.

3b. Qwen 3.7 Max (pre-registered independent-lineage replication)

5 of the 9 tasks significant; family-wise null probability $\approx 3.3 \times 10^{-5}$. **Treatment shipped zero confidently-wrong patches on every task** (gap of zero throughout — 0/10 per task, and 0/9 on the one task with an errored rollout).

Task	control	treatment	Reduction	Significance (p)
codecarbon-831	100%	0%	−100	<0.0001
kombu-2300	90%	0%	−90	0.0001
twine-1249	90%	0%	−90	0.0001
datamodel-code-generator-2408	60%	0%	−60	0.006
drf-json-api-1283	40%	0%	−40	0.043
datamodel-code-generator-2461	30%	0%	−30	0.13 (n.s.)
pycasbin-392	20%	0%	−20	0.23 (n.s.)
django-guardian-899	20%	0%	−20	0.23 (n.s.)
freezegun-582	0%	0%	0	(easiest task)

Across all runs (n=9): ships "done" but broken **50% → 0%**; fix rate **37% → 36%** (approximately flat). (Including the four large repos as a separate, navigation-confounded stratum — see §7 — adds a sixth significant task, *attrs-1448* 60%→0%; family-wise $p \approx 1.1 \times 10^{-5}$.)

3c. The two lineages compared

Both show the same effect — executable feedback drives the self-verification gap toward zero — but they are **not identical**, and the differences are informative:

- **Direction: identical.** Treatment $\leq$ control on every task in both models; treatment gap is 0% on all 9 Qwen tasks and $\leq 10\%$ on 7/9 DeepSeek tasks (the two exceptions, codecarbon and kombu, fell 100%→50%).
- **Magnitude: smaller on Qwen.** Qwen's *control* gaps are lower (e.g. guardian 100%→20%, datamodel-2461 100%→30% vs DeepSeek), because Qwen is the **stronger self-verifier** — it more often *correctly declines* to declare done when unsure. Less false confidence to remove $\Rightarrow$ a more modest (but still highly significant) effect (5/9 vs 8/9 tasks).
- **Character: diagnostic-dominant on Qwen.** Qwen's gap concentrates on hard tasks neither arm solves (twine, kombu, codecarbon), so feedback **stops confident-wrong shipping** without solving more (resolve flat). DeepSeek, the weaker blind solver, showed both diagnostic *and* generative gains (resolve doubled). Same lever, gradient expressed differently per model.

4. The pattern: a gradient, not a switch

The benefit *scales with how hard the task is*:

1. **Easy tasks** (e.g. *freezegun*): the model already gets it right blind, so running tests adds nothing — **redundant**.
2. **Medium tasks** (e.g. *pycasbin*, *drf-json-api*): the fix is reachable with iteration, so feedback drives a *run* → *see failure* → *fix* → *re-run* loop and the agent **solves more** (DeepSeek's drf went 2/10 → 10/10; Qwen's drf 6/10 → 10/10).
3. **Hard tasks** (e.g. *twine*, *kombu*, *codecarbon* — neither arm fully solves them): feedback can't conjure a fix the model can't find, but it **stops the agent declaring victory while broken** (confidently-wrong dropped to 0% for Qwen, ≤50% for DeepSeek).

Where a model sits on this gradient depends on the model: a stronger self-verifier spends more tasks in the "redundant" and "diagnostic" bands and fewer in the "generative" band. The mechanism is not "the model got smarter." The agent gained a way to **check itself against an executable definition of done** — precisely the operational case for **Harnessed Iterative SDD**: executable acceptance criteria (BDD / spec-driven) wired into the agent loop as something it *runs every iteration*, not documentation it reads.

5. Corroboration: the production tools developers use today

The two controlled models above share one scaffold (OpenHands). To check whether the *failure mode* is real beyond our scaffold, we measured the same self-verification gap on the coding agents practitioners actually run — **OpenAI Codex (gpt-5.5 , maximum reasoning effort)** and **Anthropic Claude Code (Opus 4.8)**, each via its own production CLI — on the three hardest tasks. Crucially, these tools are **execution-first**: they have a shell and can run tests. So this is *not* a controlled two-arm contrast; it is a single-condition probe (classification: *calibration*) asking one question — *do today's tools, with all their built-in execution, still ship done-but-broken against a held-out acceptance oracle?*

Task	Codex (GPT-5.5)	Claude Code (Opus 4.8)	DeepSeek control (no exec)	DeepSeek treatment (acceptance oracle)
twine-1249	100%	100%	100%	0%
guardian-899	40%	20%	100%	10%
drf-json-api-1283	60%	100%	70%	0%
mean gap	67%	73%	90%	3%

Both production lineages show a gap on all three tasks. Two findings stand out. First, **the production tools land in the same regime as the *no-execution* baseline (67–90%), not the acceptance-oracle arm (3%)** — their own shells and tests do not, by themselves, prevent confidently-wrong shipping against the true contract. Second, **raw capability does not predict the gap**: Opus was the strongest of any system on one task (guardian, 20%) yet shipped broken on *every* run of another (drf, 100%). The lever is not "can the agent run tests"; it is "can the agent run the *acceptance* definition of done" — i.e., HIT-SDD, not just having a shell.

This is corroboration that the failure mode generalizes across lineages — **not** a third causal result. It is single-condition (no no-execution arm), small (5 runs × 3 tasks), and cross-experiment (different models and scaffolds); see the limitations.

6. What this means for engineering leaders

- **Treat "the agent said done" as an unverified claim, not a result.** Our control arms declared done and were wrong 50–79% of the time — and the production tools your teams already use (Codex, Claude Code) were wrong 67–73% of the time on the same hard tasks, *despite running their own tests* (§5). Without an acceptance-execution gate, that is what reaches your review queue.
- **The highest-leverage reliability investment may be the loop, not the model.** A bigger model wasn't the variable here — *the ability to run an executable acceptance check* was, and it moved self-honesty sharply on **two independent model lineages**. Notably the *stronger* self-verifier (Qwen) still shipped confidently-wrong half the time without it.
- **This is the concrete value of executable specs in the agent era — Harnessed Iterative SDD.** Acceptance criteria stop being only human documentation and become the agent's

stopping condition, executed every iteration. Teams that already invest in executable acceptance criteria are positioned to wire them straight into agent loops.

We are running a series of conversations with engineering leaders to compare these lab findings against real-world experience — if your team has wired (or wants to wire) an executable definition of "done" into an agent loop, reach out to take part.

*A caveat to hold alongside the three points above: this is early, bounded evidence (two models, one task family). Treat it as a strong reason to **pilot** an execution-gated agent loop and measure your own "done-but-broken" rate — not yet as a settled benchmark.*

7. Limitations (these bound the claim — read them)

- **Two lineages $\Rightarrow$ replicated, not generalized.** DeepSeek V4 Pro and Qwen 3.7 Max agree, which clears a single-model fluke — but this is two models on **one task family, one substrate (SWE-bench Live, Python brownfield), one scaffold, one budget**. We make no "frontier agents in general" claim; broader task families, languages, and scaffolds remain future work.
- **The effect is model-dependent in magnitude and character.** Smaller on the stronger self-verifier (Qwen 5/9 vs DeepSeek 8/9 significant; gap 50% $\rightarrow$ 0% vs 79% $\rightarrow$ 13%); diagnostic-dominant on Qwen (resolve flat) vs diagnostic+generative on DeepSeek. A model that self-verifies near-perfectly blind would show little benefit — the value is concentrated where the model is *confidently wrong*.
- **The contrast is execution vs. no execution** — *not* vs. the agent writing and running its own tests. The control cannot run anything: a clean but conservative baseline that does not establish superiority over self-testing.
- **n = 9 of 13 certified tasks (both models).** Four large/complex repositories are deferred to a separate, navigation-confounded stratum: without a shell, the control arm cannot navigate them and stalls, so its disadvantage there is partly "can't explore the repo," not only "can't run tests." (For Qwen, one of these, *black-4684*, was further excluded for a container-infrastructure failure that left the treatment arm with no valid runs.) The reported 9 are the confound-free subset; excluding the others makes the result cleaner, not weaker.
- **Conservative oracle + a localization caveat.** The hidden oracle is the fail-to-pass acceptance subset; it likely *under-counts* the true gap (a stronger oracle with held-out regression and reproduction tests would only raise the measured effect). Separately, the treatment's `run_tests` returns named check identities, which carry partial *localization* information the control cannot get — so "executable acceptance feedback" here is the realistic, named-scenario form; an anonymized-check variant (pure pass/fail) is a planned robustness check.

- **Patch-level replay pending.** These two runs recorded per-test outcomes and patch *hashes* but not the patch text, so independent re-scoring of stored patches was not performed; determinism rests on the N=60 flake certification, and internal-consistency checks pass (243/243 Qwen records). The harness has since been fixed to persist patches, so future runs are patch-replay-valid.
- **Session-trace completeness pending.** The causal artifacts preserve run outcomes, per-test results, usage, artifact hashes, model/route metadata, and the final patch hash, but they do **not** preserve full OpenHands conversation traces, exact rendered prompts, or ordered tool-call logs as first-class artifacts. This limits independent auditability of the agent's interaction path, not the recorded scoring arithmetic. Future E2 runs require trace-complete bundles (rendered prompt, tool catalog, message/tool-call trace, tool results, final patch text, image/snapshot IDs).
- **The cross-lineage corroboration (§5) is `calibration`, not a third causal test** — single-condition, small (5 runs × 3 tasks), cross-experiment. Model identities are those resolved from the local CLIs at run time (Codex `gpt-5.5` at `xhigh`, Claude Code `opus[1m]`), not pinned dated IDs.

8. Conclusion and next steps

At brownfield scale, **executable acceptance feedback near-eliminates confidently-wrong shipping** — from **79%→13%** (DeepSeek V4 Pro) and **50%→0%** (Qwen 3.7 Max) — with the benefit scaling from redundant (easy) to generative (medium) to diagnostic (hard), and the gradient expressed differently per model. With a **pre-registered replication on a second, independent lineage now positive**, the finding moves from single-model **candidate** to **replicated** — while remaining **bounded** to this task family, substrate, scaffold, and budget. The same "done-but-broken" failure is **corroborated** on the production tools developers use today (Codex, Claude Code), which run their own tests yet still ship broken against a held-out acceptance contract (§5).

This is, concretely, evidence for the **executable-feedback core** of Harnessed Iterative Spec-Driven Development (HIT-SDD): an agent that runs an executable definition of done, every iteration, ships confidently-wrong work far less often. It is *not* yet evidence for the full HIT-SDD practice — an agent developing against an **authored** executable spec — which is the headline next step. Next steps, pre-registered and bounded: (a) the **authored-spec / OpenSpec-Gherkin** study (full HIT-SDD); (b) broaden beyond one task family / language / scaffold; (c) strengthen the oracle (held-out regression + reproduction tests) and run the anonymized-check robustness variant to separate verification from localization; (d) characterize *where* the effect concentrates (a predictive "confoundability" of tasks). We will report nulls.

Appendix A — Reproducibility & provenance

- Code and data: <https://github.com/acymment/done-but-broken> (scientific record) + <https://github.com/acymment/done-but-broken-harness> (harness). **All raw result artifacts listed below (causal pilots, contamination screens, and the §5 gap probes) live in the harness repo and are attached to its `e2-pilot-v1` release** (<https://github.com/acymment/done-but-broken-harness/releases/tag/e2-pilot-v1>), each verifiable by the SHA-256 cited here. The two *causal pilots* additionally have run-cards + `*.summary.json` in this repo's `docs/run-cards/`; the §5 corroboration runs are `calibration` and have **no run-card in this repo** (artifacts in the harness/release only).
- Harness: `done-but-broken-harness` (import package `hit_sdd_e2`) @ `f9fc5fa` (+ per-model route wiring) (Python/Docker; OpenHands agent; SWE-bench Live substrate). Scientific record repo: `done-but-broken`.
- Pre-registration: `e2-phase1-pilot-commitments-v1.md` + Addendum B (sealed 13-task certified list); analysis plan `e2-phase1-5-plan-v1.md`; **Addendum C** (sealed second-model replication: Qwen 3.7 Max route, gates, criterion). Classification: `causal_pilot`.
- Result artifacts (causal pilots):
 - DeepSeek V4 Pro: `e2-phase1-5-causal-pilot-deepseek-v4-pro.json`, SHA-256 `009b00e8c5b92b7a2f91d0a16d33847de13a1e6560daea8b24b1d6ceb6e61632`.
 - Qwen 3.7 Max: `e2-phase1-5-causal-pilot-qwen3.7-max.json`, SHA-256 `6484243829a5ee36f07dc44c7529807ced312a0f2a39b746fd804fafa4e2ce62`; run-card `docs/run-cards/e2-phase1-5-causal-pilot-qwen3.7-max-20260623.md`.
- Artifact scope: the causal JSON artifacts are score/provenance artifacts, not full session traces. They include per-run outcomes, per-test outcomes, usage, hashes, and metadata, but not complete OpenHands message/tool-call transcripts or rendered prompt packets; see §7 for the reporting limitation and the future-run fix.
- Per-model contamination screens (GATE-B), one per model: `e2-phase1-5-gateb-poolv2-new-deepseek-v4-pro-20260615-001.json` (DeepSeek, SHA `d59de9bd...`) and `e2-qwen3.7-max-contam-screen-n13-20260620-001.json` (Qwen, SHA `070d18cf...`, clean 13/13).
- Corroboration artifacts (`calibration`, §5): `e2-codex-gap-probe-20260617-001.json` (Codex `gpt-5.5 / xhigh`, SHA `7cd6a50f9628...`), `e2-claude-gap-probe-20260617-001.json` (Claude Code `opus[1m]`, SHA `8aa27c7923b3...`); 5 runs/task on the 3 hard tasks, single-condition — *not* the controlled ablation.
- Gates: contamination screen, feasibility, and N=60 flake certification each recorded as dated run-cards.
- Sanitization: git history stripped to base commit; `--network none` at run time; dependencies pre-baked at build time so suites run offline.

Working draft for discussion, not peer-reviewed. The causal figures are from two single-substrate pilots (DeepSeek, Qwen); the cross-lineage figures (§5) are single-condition calibration. All explicitly preliminary and bounded.